Lab 2: BCD Adder in Verilog

ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

Introduction

This lab looks into the basics of Verilog design by creating a BCD adder. The experiment focuses on making a 4-bit BCD adder that takes two 4-bit inputs that range from 0-9 and a carry-in, then making a two-digit BCD output (max 19). The design we will plan to use is with a standard 4-bit binary adder to make the first sum, then led next by a conversion circuit that verifies the result is represented correctly in BCD format. This is achieved by using a secondary adder to make the leftover/remainder and a multiplexer controlled by an overflow signal to select the correct BCD output. The lab also extends this concept by cascading two 4-bit BCD adders to form an 8-bit two-digit adder.

Procedure

To begin the lab, the provided 4-bit adder module from our prelabs was reviewed and reviewed by both of us. We had first started to make a 4-bit BCD adder that accepted two 4-bit inputs and a carry-in signal, producing two 4-bit BCD outputs. Two instances of the 4-bit adder were connected, the first to do the initial binary addition and the other to make the adjusted remainder when overflow occurred. A multiplexer controlled by the overflow signal was then used to determine the correct BCD output between the raw sum and the adjusted value. Once completed, the design was verified using a testbench provided, with custom input stimulus added to validate if it exceeded the total value. The design was again simulated to ensure accurate three-digit addition and validated by comparing outputs with expected BCD results. To test the code to see if it will break is when we try to see when all the numbers together would go over 19 and 199, to make sure it will fit into the boundaries.

Results

- What problems did you encounter while testing your steps yourself?
  Initially, we were unsure if our design was correct as Wolfe's testbench input where a=10 shouldn't have worked but it did. We realized this is fine and that the lab handout never explicitly said that it HAD to fail when this happened. So long as the total sum is below 19 (and 199 for the 8-bit), 4-bit inputs over 9 can work.

- Did any problems arise when demonstrating for the TA? What were they? Explain your thoughts on how/why these test cases escaped your own testing.

No, not really the only thing we ran into was adding another test bench in our step 1, and then adding the part 1 test cases to part 2.

SIMULATION RESULTS

0-9 BCD Adder:

```
[dthornburg@linux402604 Lab2]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Oct  7 17:31 2025
Start of the bcd adder test
a= 0, b= 0, cin = 0, bcd1= 0, bcd0= 0, @                    0
a= 3, b= 5, cin = 0, bcd1= 0, bcd0= 8, @                   10
a= 9, b= 5, cin = 0, bcd1= 1, bcd0= 4, @                   20
a= 3, b= 9, cin = 1, bcd1= 1, bcd0= 3, @                   30
a=10, b= 5, cin = 1, bcd1= 1, bcd0= 6, @                   40
a= 9, b= 9, cin = 1, bcd1= 1, bcd0= 9, @                   50
End of the bcd adder test
$finish called from file "bcd_tb.v", line 25.
$finish at simulation time                  60
           V C S   S i m u l a t i o n   R e p o r t
Time: 60
CPU Time:      0.190 seconds;       Data structure size:   0.0Mb
Tue Oct  7 17:31:10 2025
```

0-99 BCD Adder:

```
[dthornburg@linux402604 Lab2]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Oct  7 17:37 2025
Start of the bcd 3 digit adder test
a1= 0, a0= 0, b1= 0, b0= 0, cin = 0, bcd2= 0, bcd1= 0, bcd0= 0, @                    0
a1= 0, a0= 3, b1= 0, b0= 5, cin = 0, bcd2= 0, bcd1= 0, bcd0= 8, @                   10
a1= 0, a0= 9, b1= 0, b0= 5, cin = 0, bcd2= 0, bcd1= 1, bcd0= 4, @                   20
a1= 0, a0= 3, b1= 0, b0= 9, cin = 1, bcd2= 0, bcd1= 1, bcd0= 3, @                   30
a1= 0, a0=10, b1= 0, b0= 5, cin = 1, bcd2= 0, bcd1= 1, bcd0= 6, @                   40
a1= 0, a0= 9, b1= 0, b0= 9, cin = 1, bcd2= 0, bcd1= 1, bcd0= 9, @                   50
a1= 3, a0= 7, b1= 1, b0= 5, cin = 0, bcd2= 0, bcd1= 5, bcd0= 2, @                   60
a1= 3, a0= 7, b1= 8, b0= 9, cin = 0, bcd2= 1, bcd1= 2, bcd0= 6, @                   70
a1= 3, a0= 7, b1= 1, b0= 5, cin = 1, bcd2= 0, bcd1= 5, bcd0= 3, @                   80
a1=12, a0= 9, b1= 7, b0= 9, cin = 1, bcd2= 1, bcd1=10, bcd0= 9, @                   90
a1= 9, a0= 9, b1= 9, b0= 9, cin = 1, bcd2= 1, bcd1= 9, bcd0= 9, @                  100
a1= 9, a0= 9, b1= 9, b0= 9, cin = 0, bcd2= 1, bcd1= 9, bcd0= 8, @                  110
End of the bcd adder test
$finish called from file "bcd_tb2.v", line 37.
$finish at simulation time                 120
           V C S   S i m u l a t i o n   R e p o r t
Time: 120
CPU Time:      0.190 seconds;       Data structure size:   0.0Mb
Tue Oct  7 17:37:29 2025
[dthornburg@linux402604 Lab2]$
```

Conclusion

Overall, in this lab we created, designed, and implemented our BCD adder. By combining two 4-bit adders, we were able to design a circuit that manages decimal addition and makes accurate BCD outputs. Using the overflow detection and a multiplexer ensured that the output was changed when the binary sum exceeded nine. Through simulation and testing, we verified that both the single-digit and two-digit BCD adders functioned as expected. All inputs were correctly summed up, and the outputs never exceeded either 19 or 199.

Appendix

Place code and other data here.

CODE

```
module bcdadd (
        input [3:0] A,
        input [3:0] B,
        input Ci,
        output [3:0] bcd0,
        output [3:0] bcd1
);
        wire [3:0] S;
        wire Cout;
        wire [3:0]addr2out;
        wire C_rem_out;
        adder4 one (A,B,Ci,Cout,S);
        adder4 two (4'b0110,S,4'b0000,C_rem_out, addr2out);
        wire ovf_out;
        assign ovf_out = Cout || (S > 4'b1001);
        assign bcd0 = ovf_out ? addr2out : S;
        assign bcd1 = {3'b000, (C_rem_out | Cout)}; // | Cout is necessary
endmodule
```

```
module testbench();

reg [3:0] a,b;
reg c;
wire [3:0] bcd0, bcd1;

bcdadd dut(a, b, c, bcd0, bcd1);
```

```
initial begin
        $display("Start of the bcd adder test");
        $monitor("a=%d, b=%d, cin = %d, bcd1=%d, bcd0=%d, @ %d", a, b, c, bcd1,
bcd0, $time);
a=4'b0; b=4'b0; c = 0;
#10
a=4'd3; b=4'd5;
#10
a=4'd9; b=4'd5;
#10
a=4'd3; b=4'd9; c=1;
#10
a=4'd10; b=4'd5;
#10
a=4'd9; b=4'd9;
#10
$display("End of the bcd adder test");
$finish;

end

endmodule
```

```
module bcdadd2 (
        input [3:0] A1,         //tens
        input [3:0] A0, //ones
        input [3:0] B1, //tens
        input [3:0] B0, //ones
        input Ci,
        output [3:0] bcd0,
        output [3:0] bcd1,
        output [3:0] bcd2
);
        wire [3:0] ones_bcd0;
        wire [3:0] ones_bcd1;
        bcdadd b_one (A0, B0, Ci, ones_bcd0, ones_bcd1);
        wire [3:0] tens_bcd0;
        wire [3:0] tens_bcd1;
        bcdadd b_two (A1, B1, ones_bcd1[0], tens_bcd0, tens_bcd1);
        assign bcd0 = ones_bcd0;
        assign bcd1 = tens_bcd0;
        assign bcd2 = tens_bcd1;
endmodule
```

```
module testbench2();
```

```
reg [3:0] a1,a0,b1,b0;
reg c;
wire [3:0] bcd0, bcd1, bcd2;

bcdadd2 dut(a1, a0, b1, b0, c, bcd0, bcd1, bcd2);

initial begin
        $display("Start of the bcd 3 digit adder test");
        $monitor("a1=%d, a0=%d, b1=%d, b0=%d, cin = %d, bcd2=%d, bcd1=%d,
bcd0=%d, @ %d", a1, a0, b1, b0, c, bcd2, bcd1, bcd0, $time);
a1=4'b0; a0=4'b0; b1=4'b0; b0=4'b0; c = 0;
#10
a1=4'd0; a0=4'd3; b1=4'd0; b0=4'd5;
#10
a1=4'd0; a0=4'd9; b1=4'd0; b0=4'd5;
#10
a1=4'd0; a0=4'd3; b1=4'd0; b0=4'd9; c=1;
#10
a1=4'd0; a0=4'd10; b1=4'd0; b0=4'd5;
#10
a1=4'd0; a0=4'd9; b1=4'd0; b0=4'd9;
#10
a1=4'd3; a0=4'd7; b1=4'd1; b0=4'd5; c=0;
#10
a1=4'd3; a0=4'd7; b1=4'd8; b0=4'd9;
#10
a1=4'd3; a0=4'd7; b1=4'd1; b0=4'd5; c=1;
#10
a1=4'd12; a0=4'd9; b1=4'd7; b0=4'd9;
#10
a1=4'd9; a0=4'd9; b1=4'd9; b0=4'd9;
#10
a1=4'd9; a0=4'd9; b1=4'd9; b0=4'd9; c=0;
#10
$display("End of the bcd adder test");
$finish;

end

endmodule
```